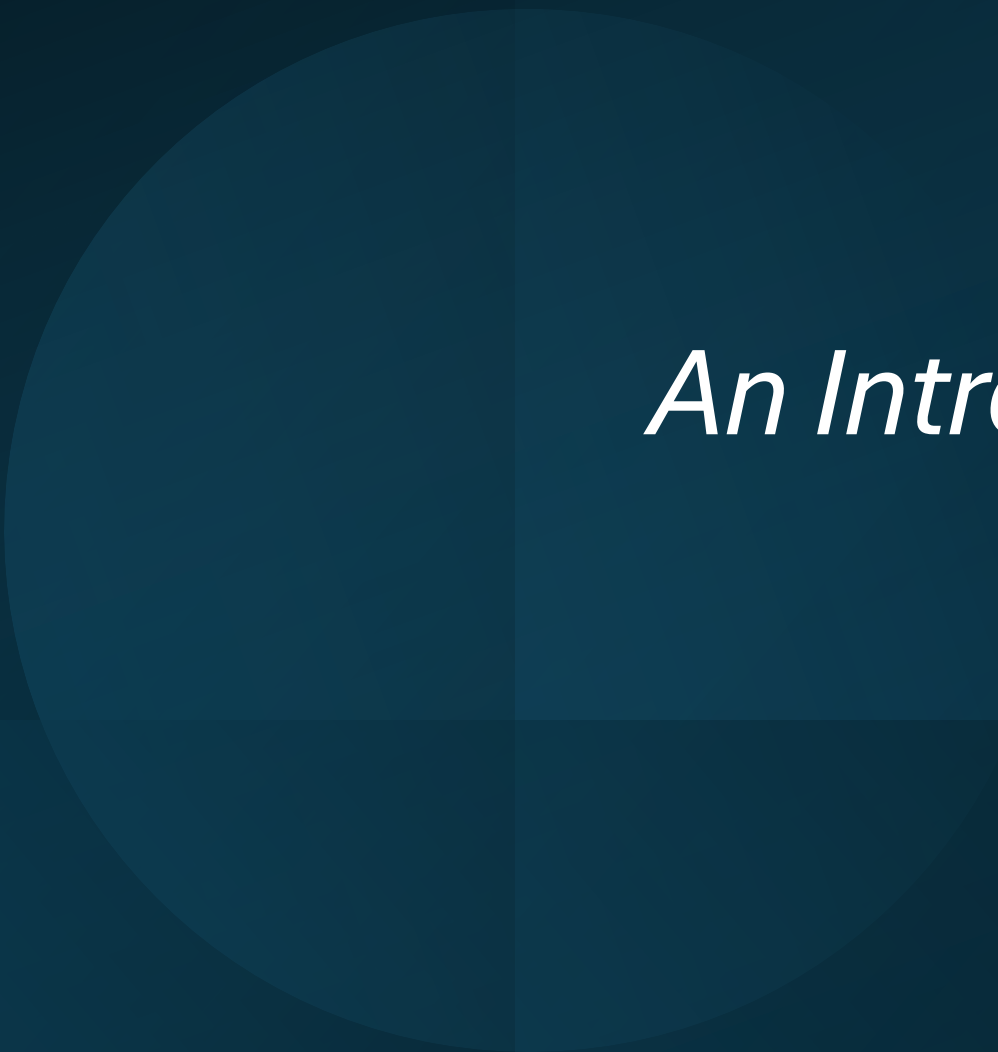




# *An Introduction to LLMs*

# What is a Large Language Model (LLM)?

It is a (Deep Neural Network based) model designed to generate the next word in a sentence.



Examples: Open AI's GPT, Google's BERT, Meta's Llama, etc.



In particular, LLMs are auto-regressive next token predictors.



Equivalently, it is model designed to translate a given sentence (in English) into a sentence in a different language (French).

# Auto- Regressive?

## Auto-Regressive

This means that the model generates output **one token at a time**, and **each new token is based on all the previous ones**.

So, if you're typing:

| "The cat sat on the"

The model will:

1. Look at all the tokens so far: "The", "cat", "sat", "on", "the"
  2. Predict the most likely next token: "mat"
  3. Add "mat" to the sequence and repeat the process if more tokens are needed.
-

# Next Word (token) Prediction?

## Next-Token Prediction

The model's core task is:

Given a sequence of tokens, predict the next token.

It doesn't think in full sentences or paragraphs at once. Instead:

- It calculates a probability for every possible token that could come next.
- It picks the token with the highest probability (or samples based on the distribution).

 For example:

Given: "I want to eat a"

Possible predictions might be:

- "pizza" (60% probability)
- "burger" (20%)
- "sandwich" (10%)
- "car" (0.01%)

It will likely choose "pizza" .

# Translation as a Next Token Prediction Problem

## 💡 Key idea:

The model doesn't "translate" in a traditional sense. It treats the task as:  
"Given an input prompt, predict the next token... repeatedly."

## 📌 Step 1: Input as Context

You give the model a **prompt** like:

mathematica

Translate to French: I love Yale

This entire line becomes the **context**.

The model starts generating tokens **after the colon**, one at a time.

# Translation as a Next Token Prediction Problem....

## ⚙️ Step 2: Predicting the Next Token

Now the model begins predicting:

1. First prediction: "J" (very likely)
2. Then: "'"
3. Then: "aime"
4. Then: " "
5. Then: "Yale"
6. Then: "<eos>" (end-of-sentence token)

Each token is chosen based on:

All previous tokens: "Translate to French: I love Yale J'ai" → next likely token is "me"

# Large Language Models: Why Large?

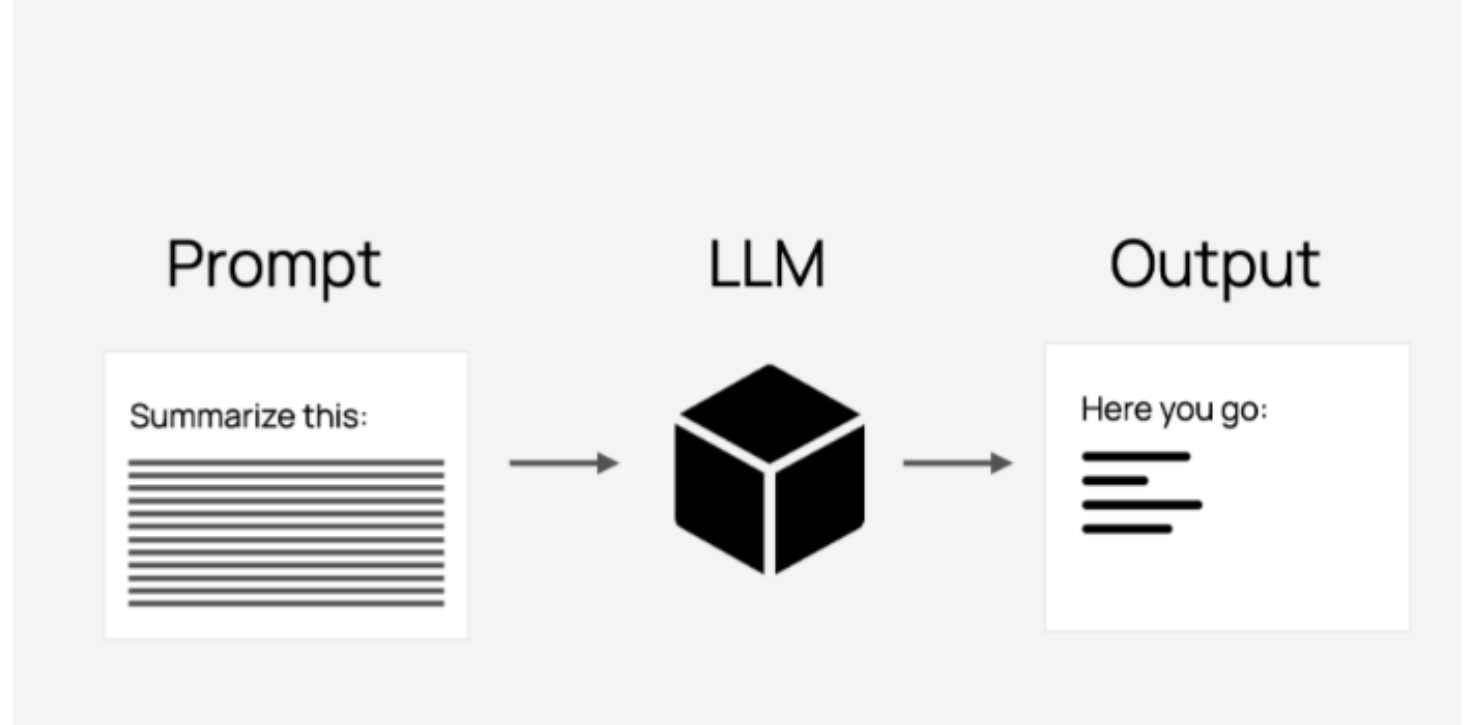
- Trained on massive amount of data; contains millions/billions of parameters; training requires massive amount of compute (GPUs)

## ChatGPT: Parameters, data size, and computing resources

Here's a comparison table summarizing the parameters, data size, and computing resources for ChatGPT models:

| Aspect  | ChatGPT (based on GPT-3.5)                                                                                                                                 | GPT-4                                                                                                            | ChatGPT-4o               |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------|
| Parameters                                                                                 | 175 billion                                                                                                                                                | 1.76 trillion (reportedly 8 models, each with 220 billion parameters)                                            | 1.7 trillion             |
| Training Data Size                                                                         | 570GB of text data, including books, Wikipedia, research journals, web articles, etc.                                                                      | Estimated 13 trillion tokens (around 10 trillion words)                                                          | Not officially disclosed |
| Computing Resources (Estimates)                                                            | Training: 1000 NVIDIA V100 GPUs, potentially costing \$100 million. Running: Estimated cost of \$700,000 per day, with each query costing around \$0.0036. | Training: Reportedly over \$100 million. Requires significant computational power due to larger parameter count. | Not officially disclosed |

# Large Language Models: Why Language?



- Inputs and Outputs are both Language, in particular, text data.
- Additionally, these models are used to do a wide variety of Natural Language Processing (NLP) tasks such as translation, sentiment analysis, etc.





# Why LLMs are so popular?

## LLMs vs earlier NLP models

- LLMs can perform **very well on a wide variety of NLP tasks** such as translation, human like text generation, drafting emails, etc.
- Earlier NLP models (such as RNNs, LSTM Models, etc.) were primarily designed for a specific task.
- Let's try ChatGPT

# Why LLMs are so good?

---

- What is the secret sauce ?







THIS IS NOT WAR,  
IT'S EXTINCTION

Answer:  
Transformers?

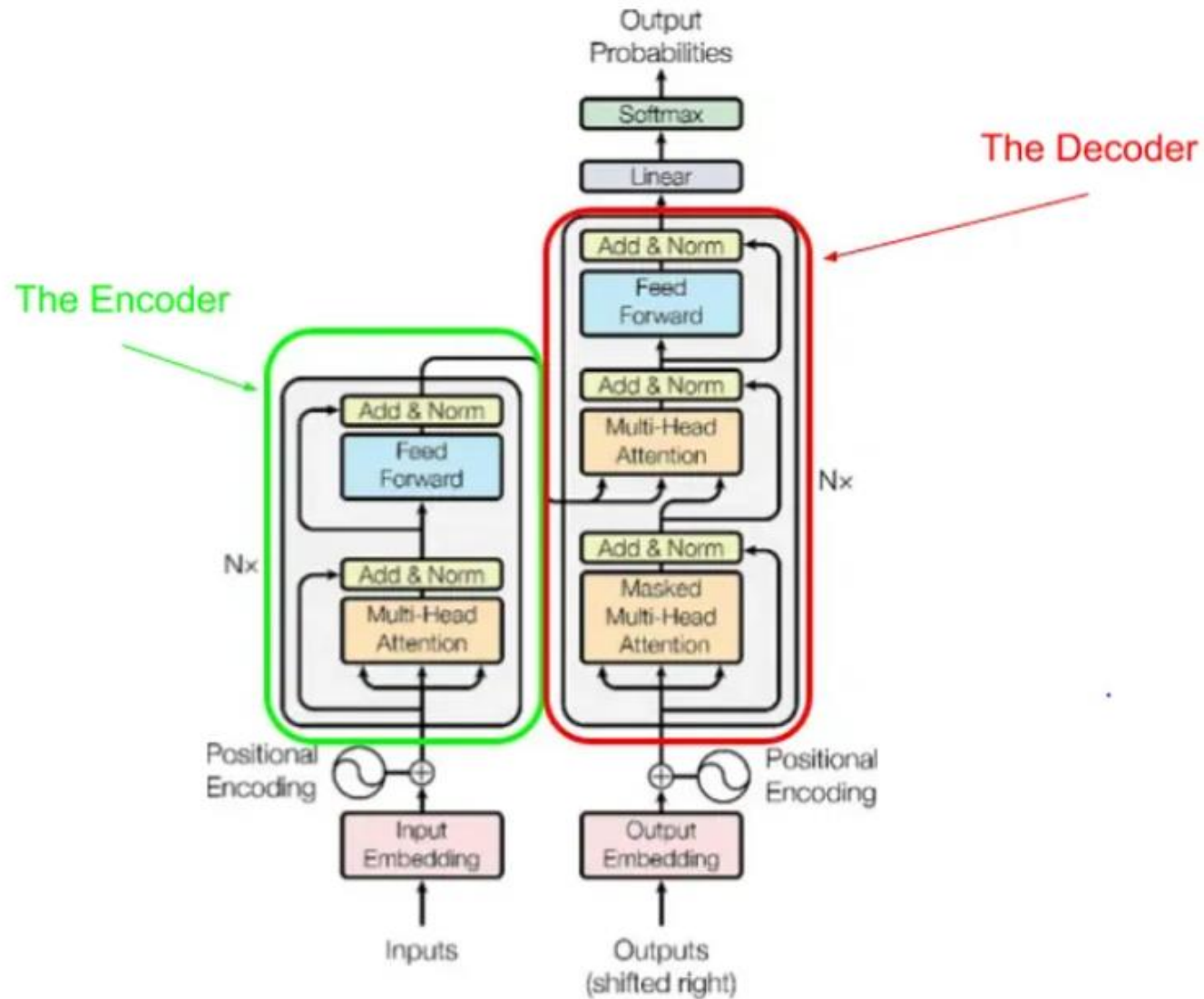
A MICHAEL BAY FILM  
**TRANSFORMERS**  
AGE OF EXTINCTION

PARAMOUNT PICTURES PRESENTS IN ASSOCIATION WITH HASBRO & BONAVENTURA PICTURES PRODUCTION A TOM DESANTO/DON MURPHY PRODUCTION  
AN IAN BRYCE PRODUCTION A MICHAEL BAY FILM MARK WAHLBERG "TRANSFORMERS: AGE OF EXTINCTION" STANLEY TUCCI VISUAL EFFECTS INDUSTRIAL LIGHT & MAGIC MUSIC BY STEVE JABLONSKY  
EDITED BY WILLIAM GOLDBERG A.C.E. ROGER BARTON PAUL RUBELL A.C.E. PRODUCTION DESIGNER JEFFREY BEECROFT DIRECTOR OF PHOTOGRAPHY AMIR MOKRI EXECUTIVE PRODUCERS STEVEN SPIELBERG MICHAEL BAY BRIAN GOLDNER MARK VAHRADIAN  
PRODUCED BY DON MURPHY & TOM DESANTO LORENZO DI BONAVENTURA IAN BRYCE BASED ON HASBRO'S TRANSFORMERS ACTION FIGURES WRITTEN BY EHREN KRUGER DIRECTED BY MICHAEL BAY

IN THEATRES, REALD 3D AND IMAX 3D

COMING SOON

Actually, our  
Transformers  
look like this



Transformer  
Architecture was first  
introduced in this (now  
famous) 2017 paper.

The Big Bang of NLP  
(and in general of ML/AI  
research).

---

## Attention Is All You Need

---

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

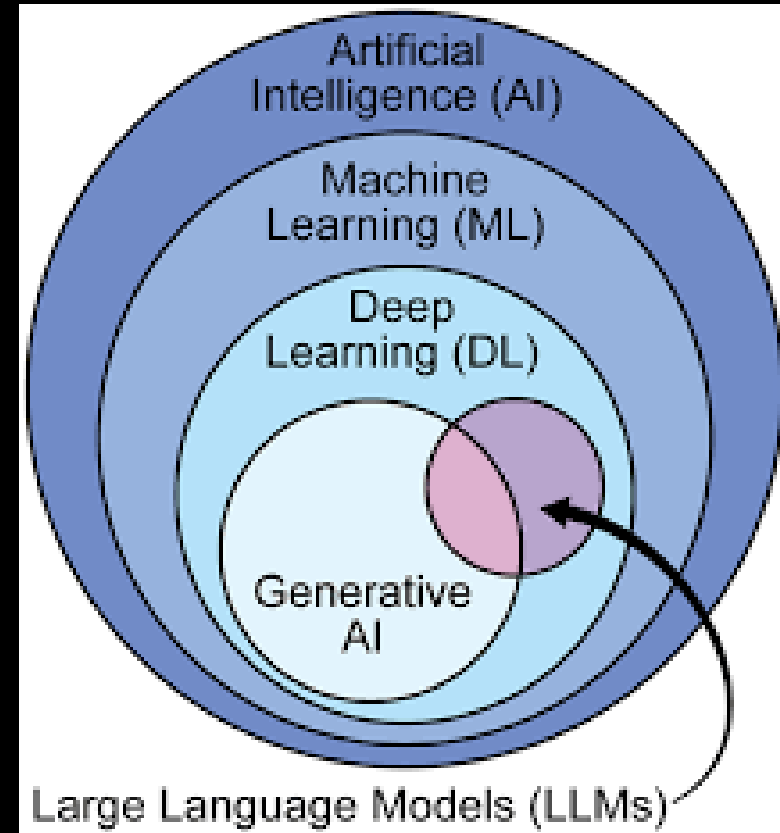
**Aidan N. Gomez\* †**  
University of Toronto  
aidan@cs.toronto.edu

**Łukasz Kaiser\***  
Google Brain  
lukaszkaizer@google.com

**Illia Polosukhin\* ‡**  
illia.polosukhin@gmail.com

# LLMs vs Generative AI vs Deep Learning

- LLMs leverages Gen AI to process and generate human like text.
- Given a user query  $x \in S$ , we want to consider all possible replies  $s \in S$  and generate a reply such that  $P(s | x)$  is maximized.



# What about ChatGPT?

---

- Is ChatGPT also learning how to generate  $x$  from  $P(x)$ ?
- Yes.
- In this case,  $z$  is the user input query. e.g. “How can I code a for loop in Python”
- $x \sim p(x | z)$  is sampled from the space of all possible responses.
- Given a user’s input sentence, we want to map it to the most likely response out of all possible responses.

# Applications



Content Creation/Novel Text Creation



Chatbots/Virtual Assistants



Sentiment Analysis



Text summarization



Machine Translation



And many more



# Transformers

A closer look

As we talked earlier:  
Transformer  
Architecture was first  
introduced in this (now  
famous) 2017 paper.

The Big Bang of NLP  
(and in general of ML/AI  
research).

---

## Attention Is All You Need

---

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

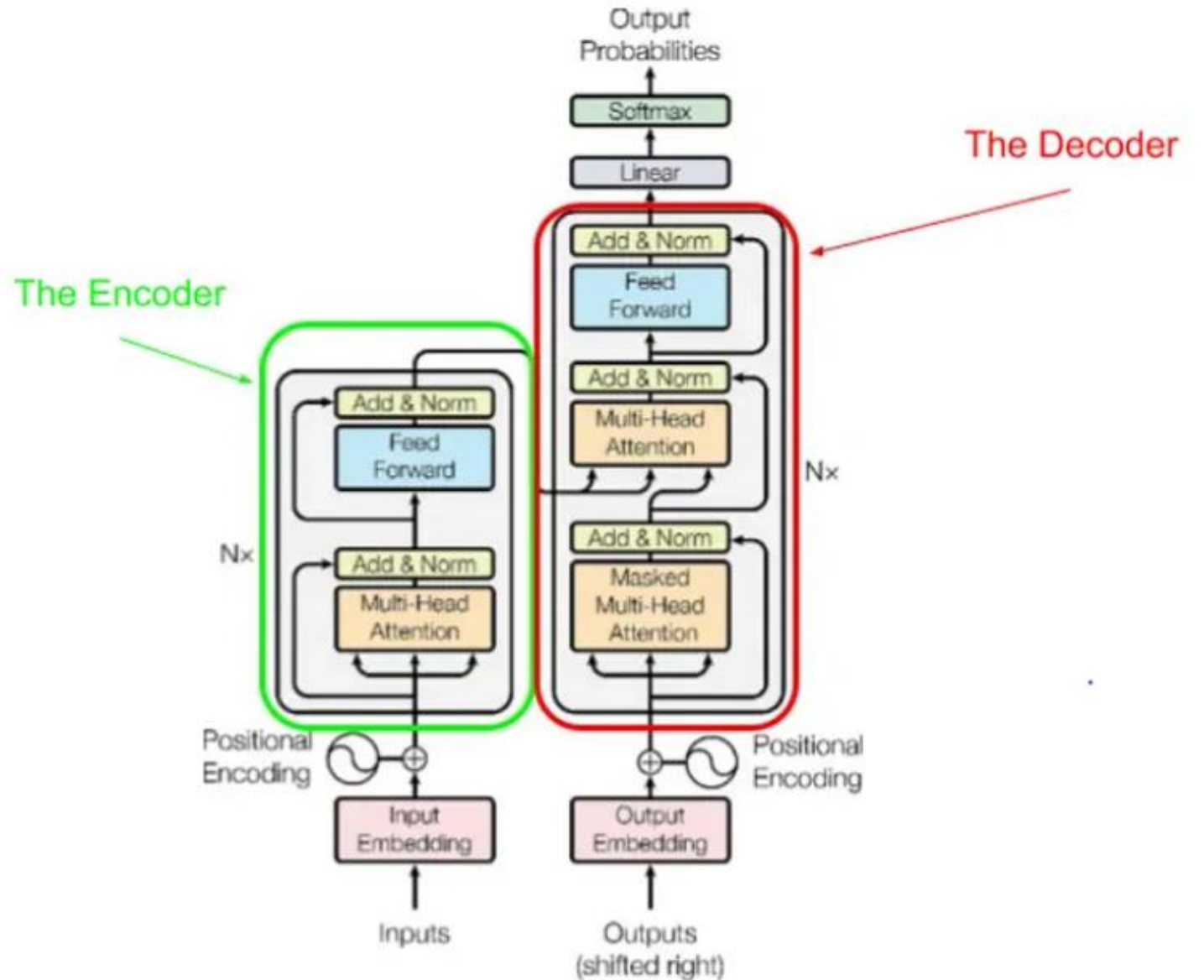
**Aidan N. Gomez\* †**  
University of Toronto  
aidan@cs.toronto.edu

**Łukasz Kaiser\***  
Google Brain  
lukaszkaizer@google.com

**Illia Polosukhin\* ‡**  
illia.polosukhin@gmail.com

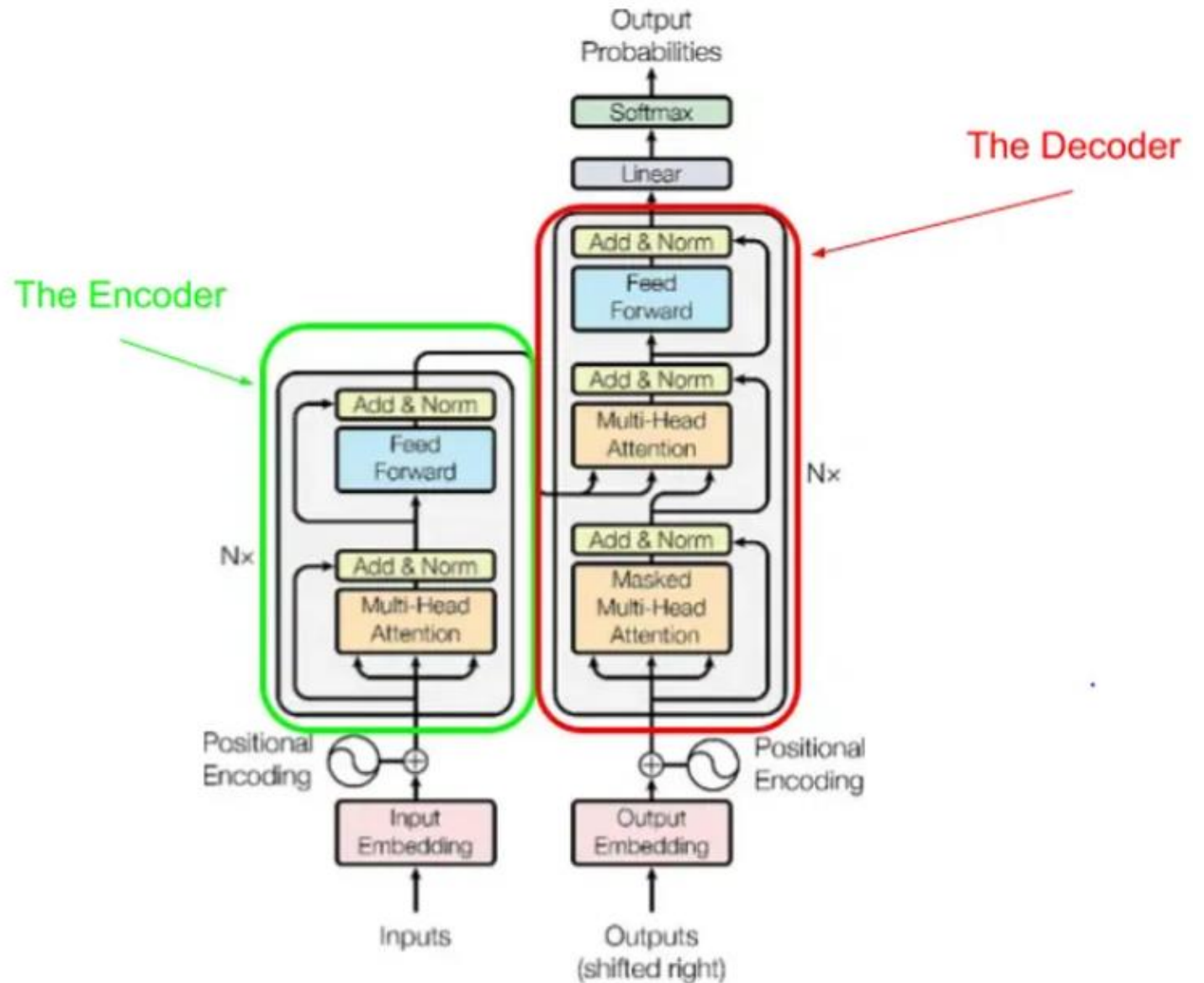
On the right, you see the exact Transformer Architecture introduced in the paper Vaswani et al.

This Transformer is credited as the mother of all transformers. In particular, Open AI's GPT and Google's BERT are special (simpler) cases of the transformer on the right.

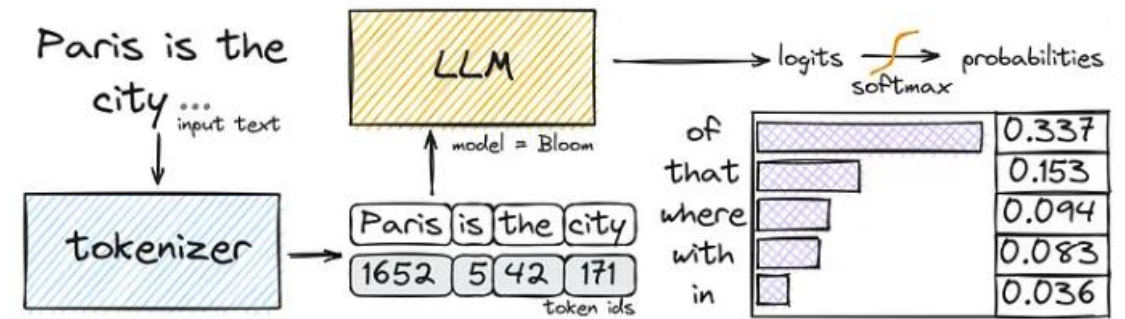
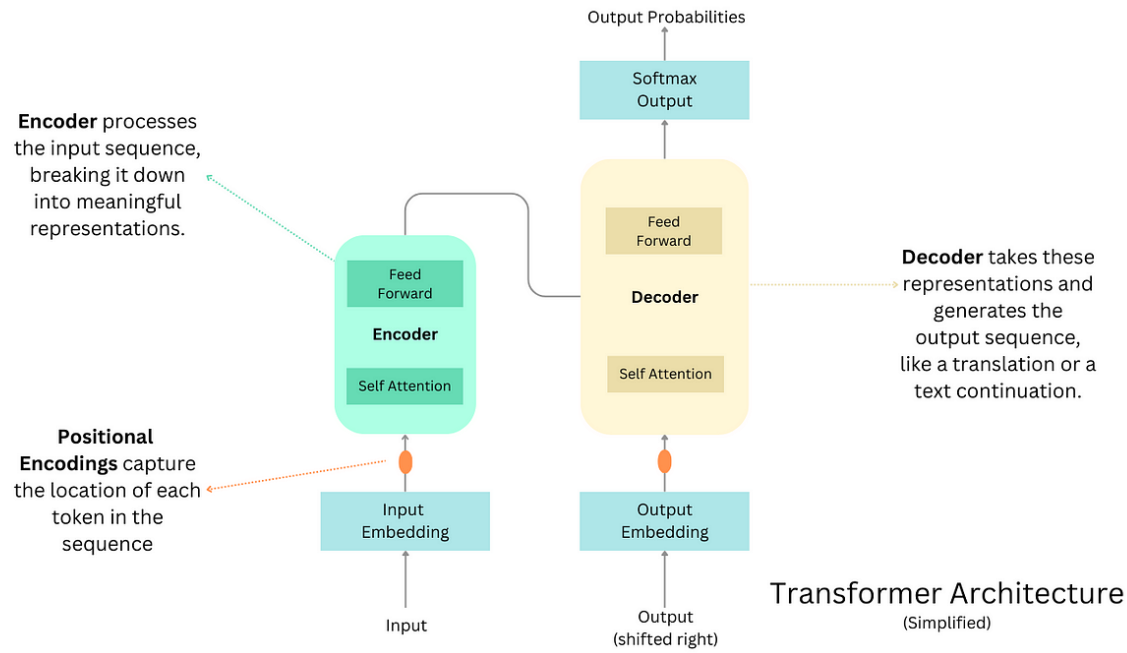


This Transformer model was developed for machine translation problem, for example from English to French.

Can you guess the number of citations for this paper?



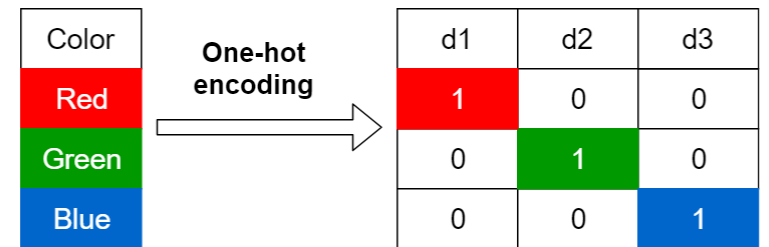
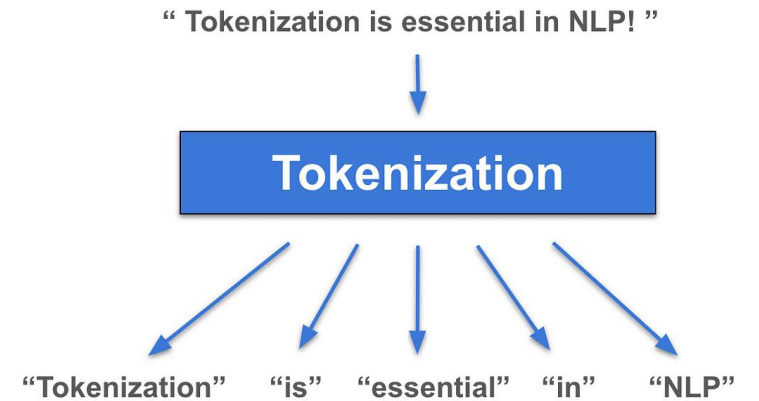
# A toned-down version of Transformer Architecture Schematic



# Step 0: Tokenization

---

- Tokenization is usually followed by some form of vector mapping. For example, One-hot encoding.
- There is a huge literature on Tokenization itself.
- Once we have the words (tokens) in the vectors form, we can input them into the encoder block of the Transformer.



# Encoder



Goal of the Encoder is to map a sequence of words (Paris is the city) to a sequence of vectors such that the semantic meaning between words is well preserved.



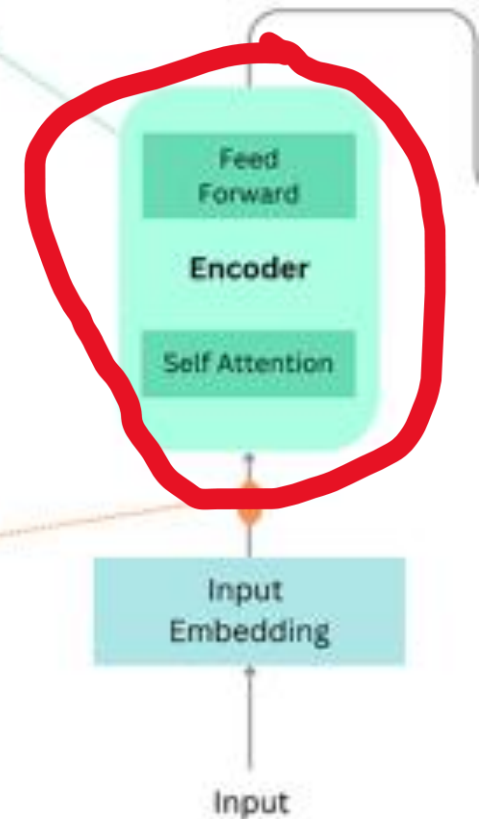
As we go higher up in an encoder, richer and more refined the vector embeddings.



Once we have these embeddings, we can then feed these to the decoder block for various applications.

**Encoder** processes the input sequence, breaking it down into meaningful representations.

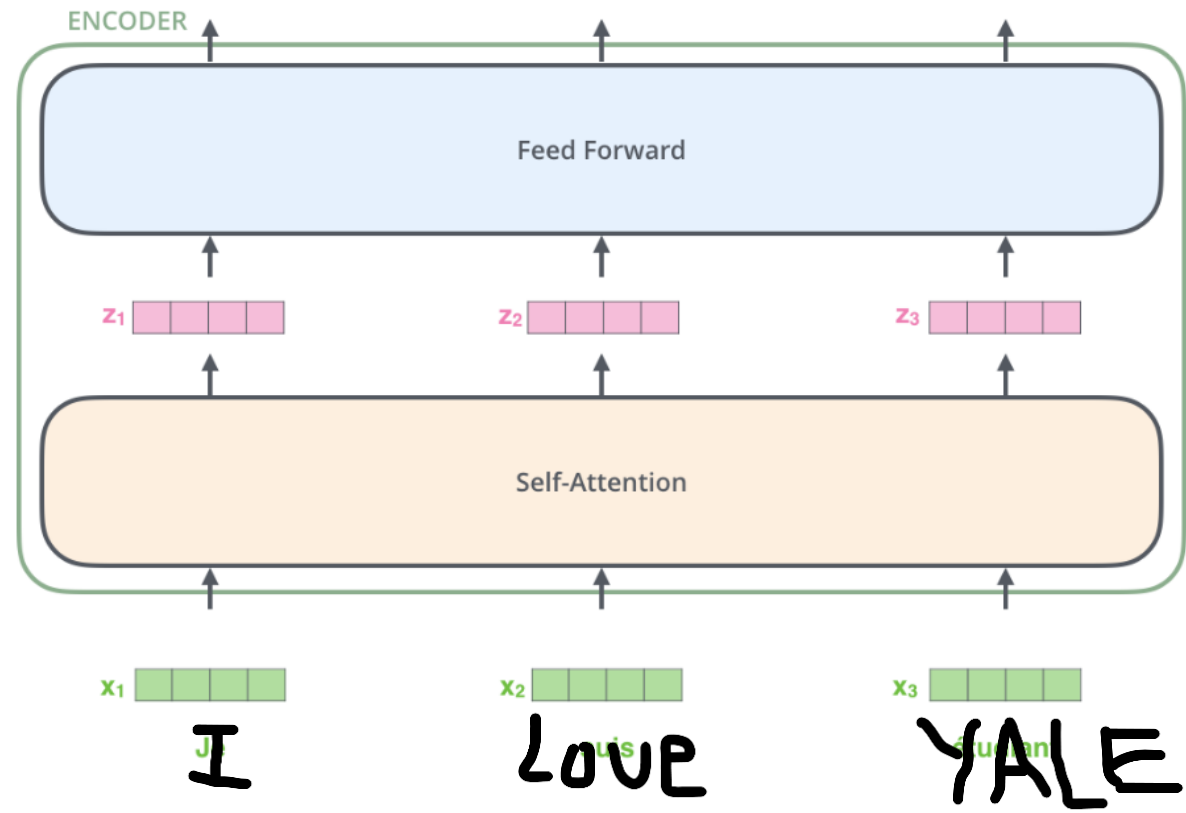
**Positional Encodings** capture the location of each token in the sequence





An encoder receives a list of vectors as input.

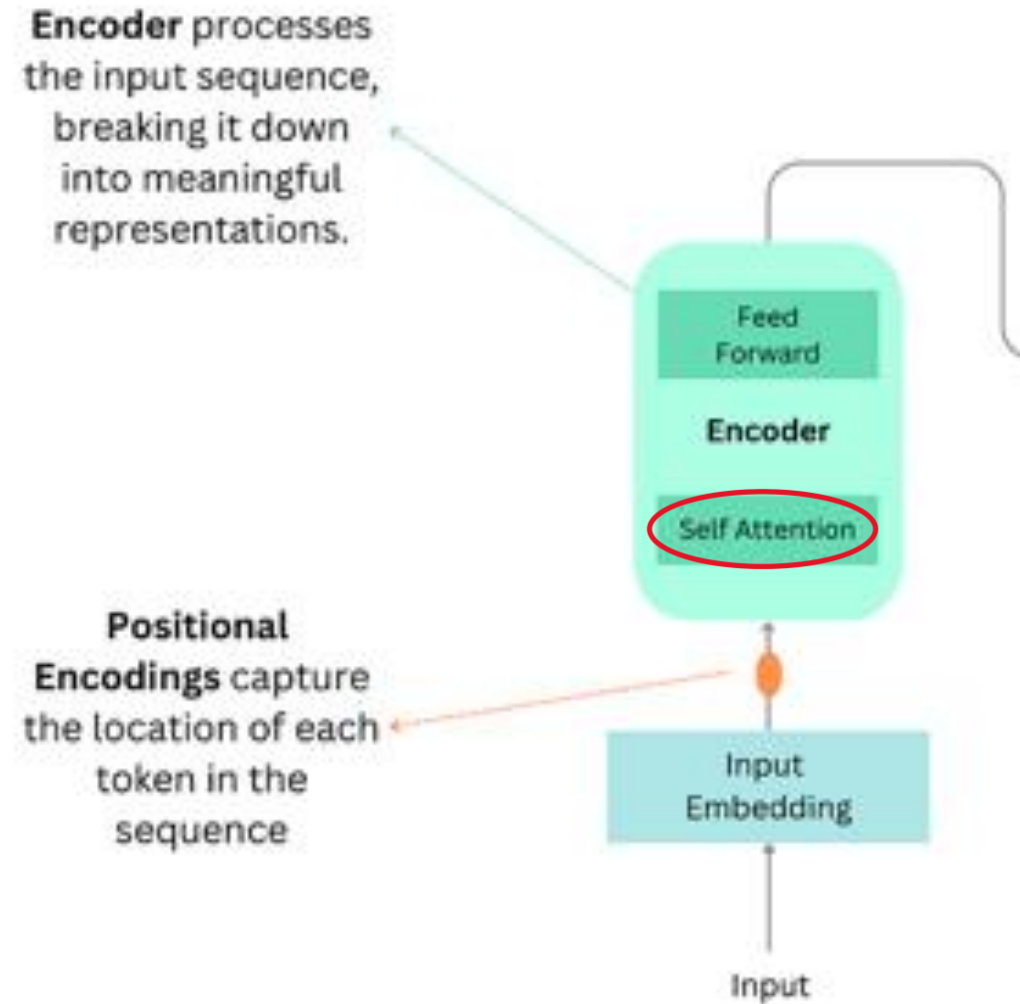
It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural network, then sends out the output upwards to the next encoder.





Attention Layer is important and allows Transformers to model long run dependencies among words (tokens) and hence bake context into the vector embeddings.

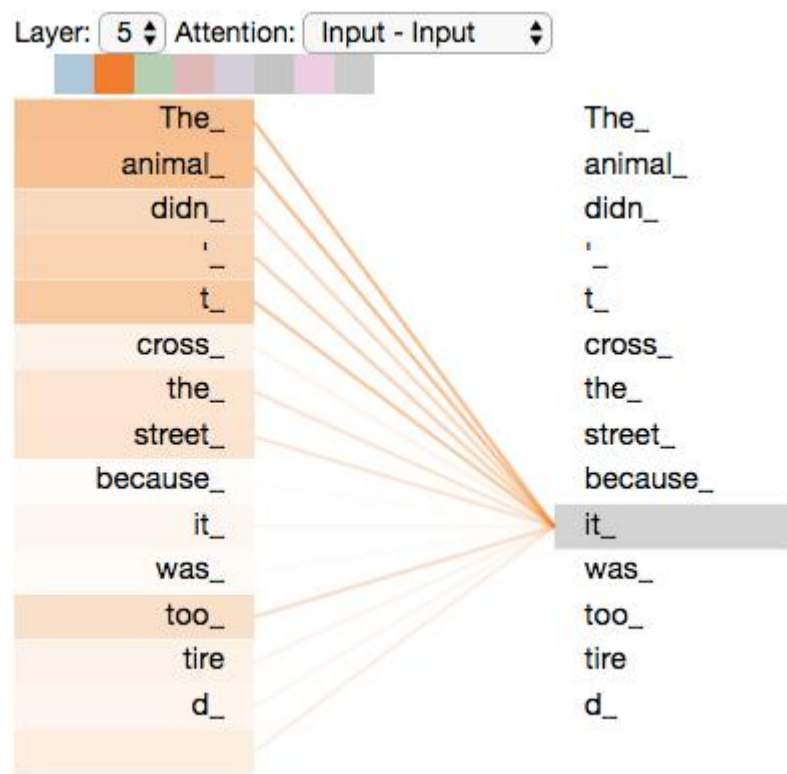
---



# Attention Example

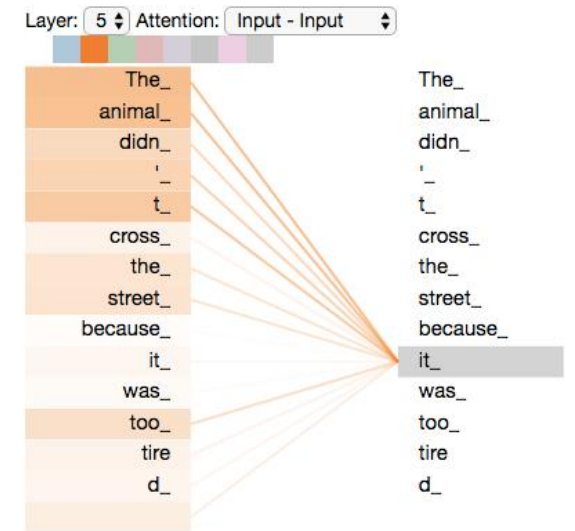
Say the following sentence is an input sentence we want to translate:

"The animal didn't cross the street because it was too tired"



# Attention Example

- What does “it” in this sentence refer to? Is it referring to the street or to the animal? It’s a simple question to a human, but not as simple to an algorithm.
- When the model is processing the word “it”, self-attention allows it to associate “it” with “animal”.
- As the model processes each word (each position in the input sequence), self attention allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.



Say the following sentence is an input sentence we want to translate:

"The animal didn't cross the street because it was too tired"

# Attention

---

Allows model to weigh importance of different words/tokens relative to each other.

---

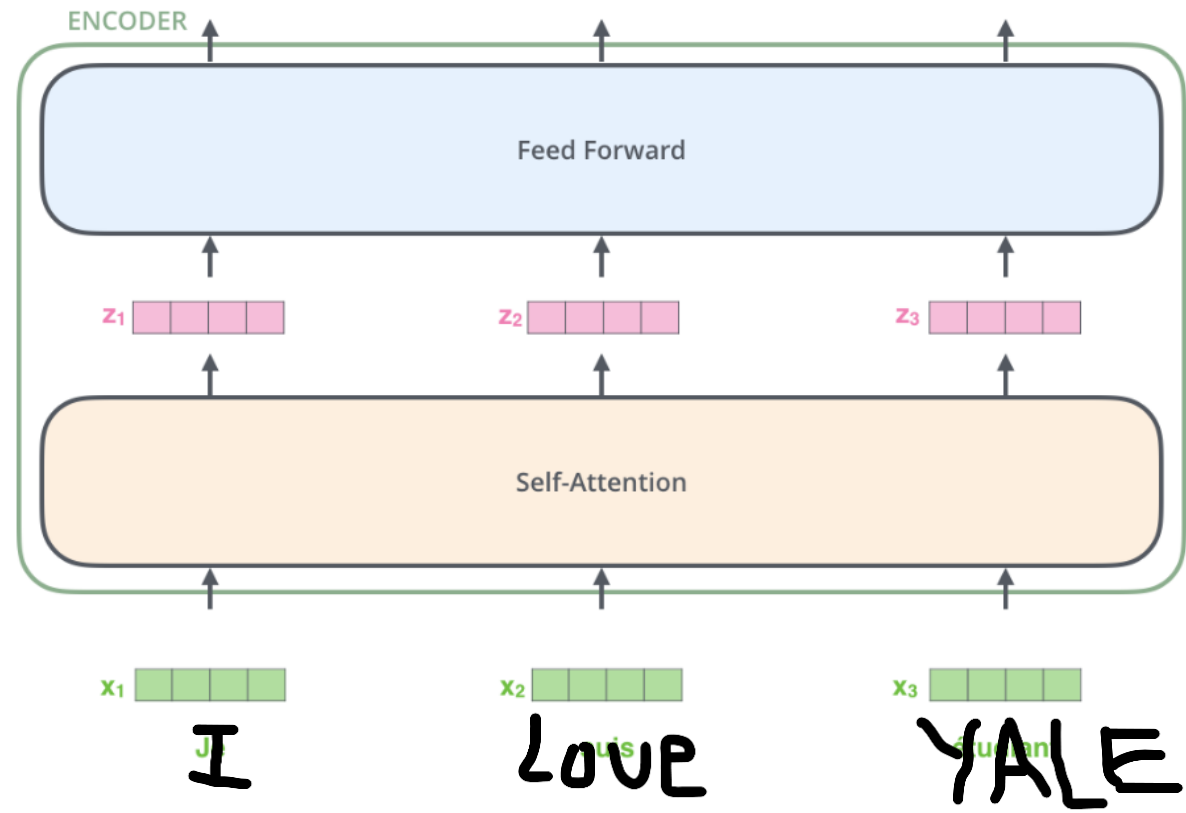
Enables model to capture long range dependencies.

---

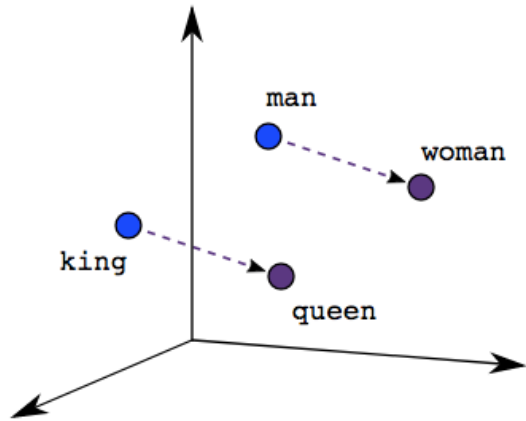
Allowing the model to learn context.

An encoder receives a list of vectors as input.

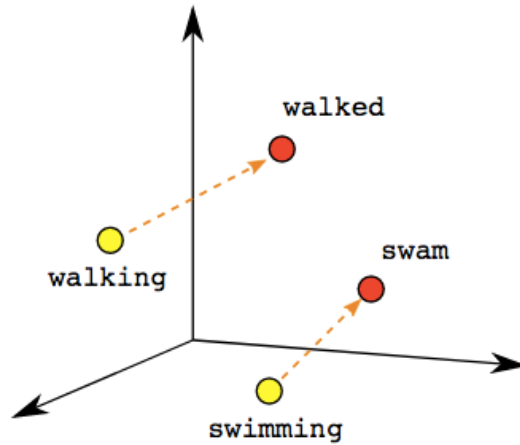
It processes this list by passing these vectors into a 'self-attention' layer, then into a feed-forward neural network, then sends out the output upwards to the next encoder.



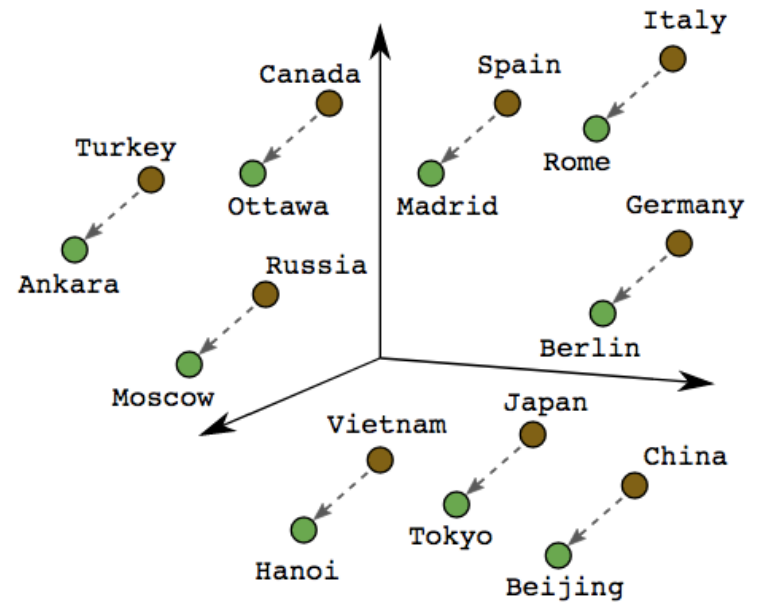
# Vector Embeddings



Male-Female

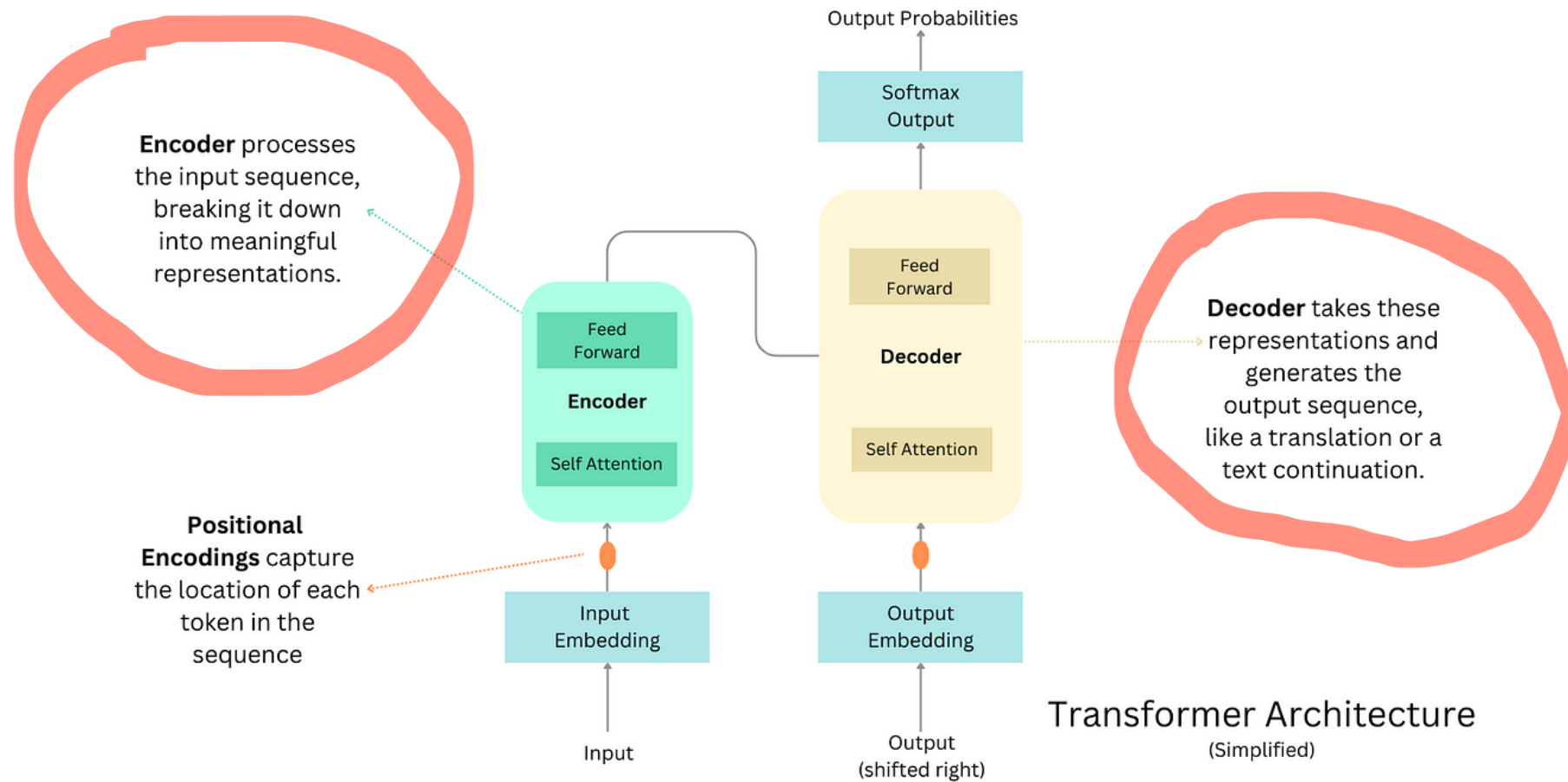


Verb Tense



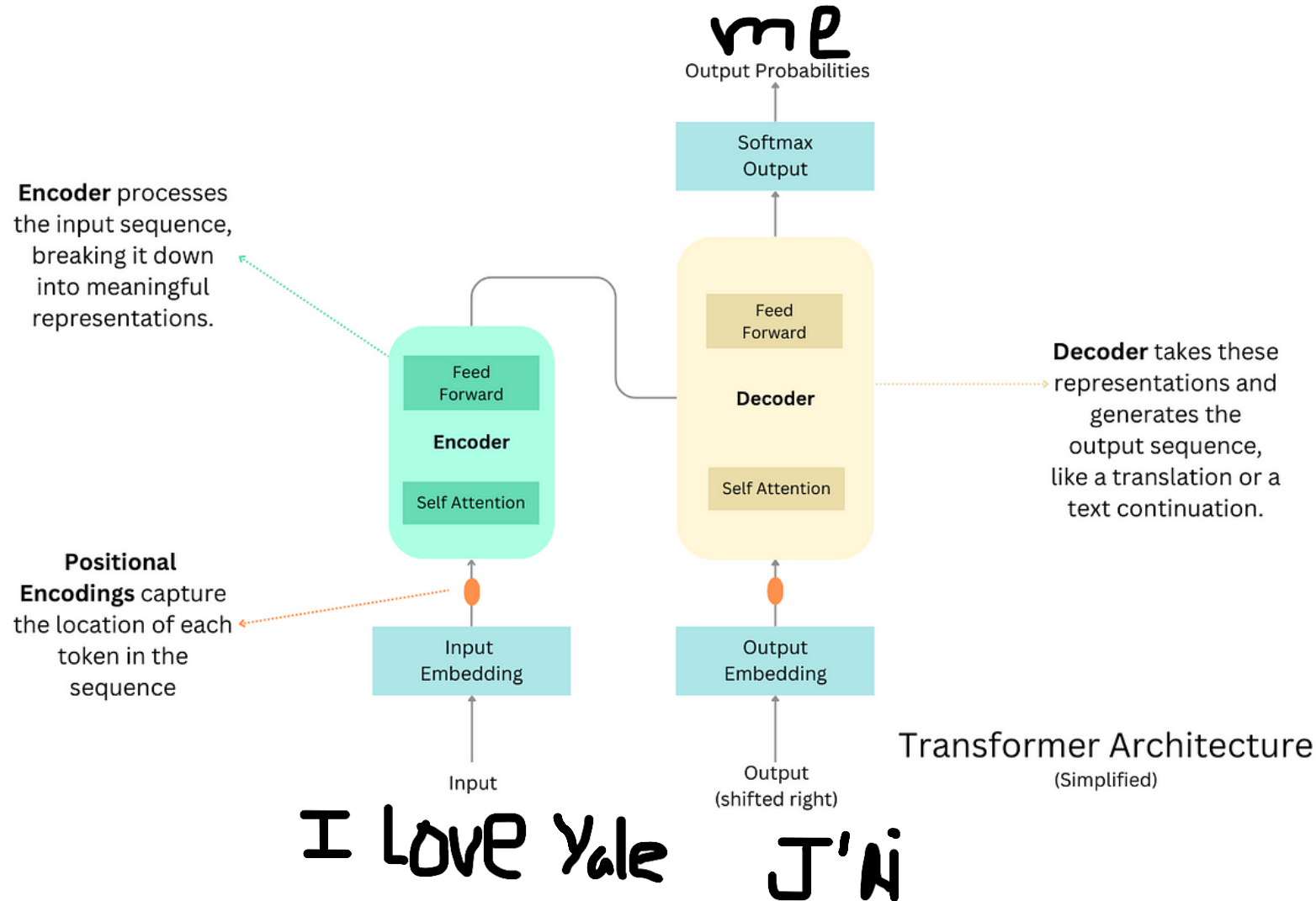
Country-Capital

# Finally, these representations are then fed to the Decoder part

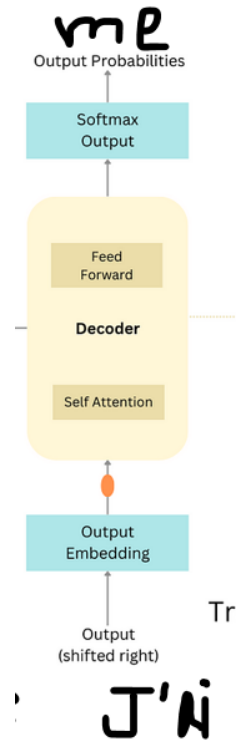




Decoder uses both the representation as well as partially decoded/translated sentence as inputs



# Auto-Regressive Prediction



Now the model begins predicting:

1. First prediction: "J" (very likely)
2. Then: ""
3. Then: "aime"
4. Then: ""
5. Then: "Yale"
6. Then: "<eos>" (end-of-sentence token)

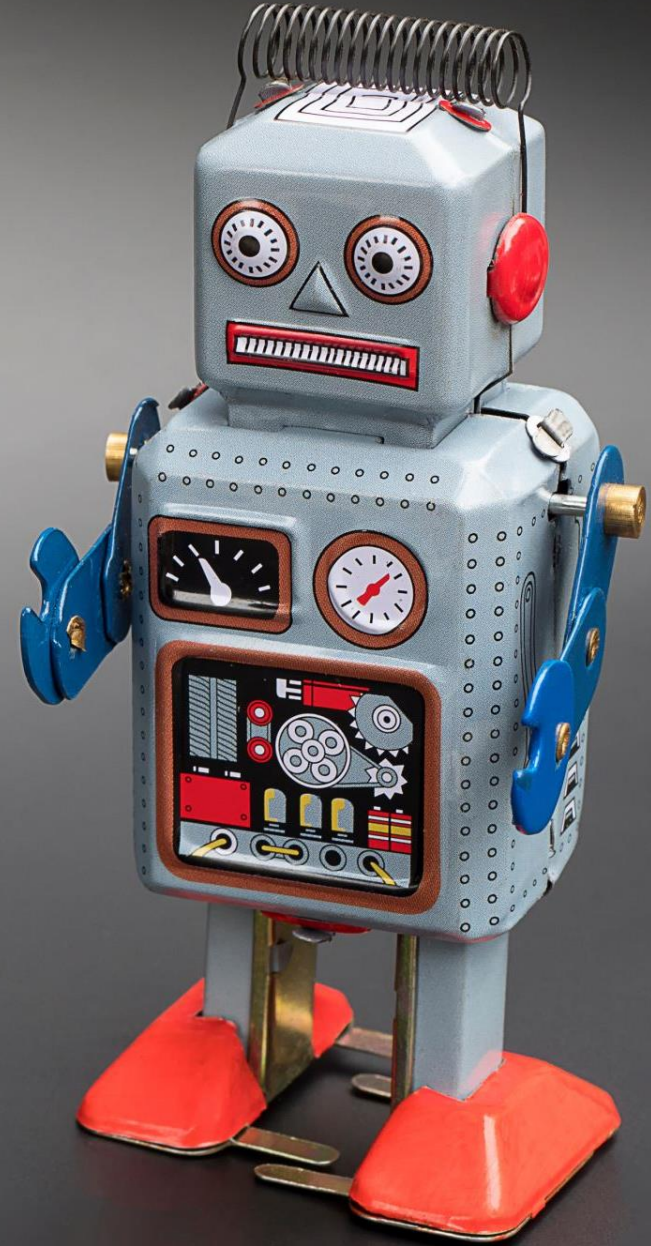
Each token is chosen based on:

All previous tokens: "Translate to French: I love Yale J'ai" → next likely token is "me"

# Encoder vs Decoder

---

1. Transformers may only have a decoder block, for example, Open AI's GPT (Generative Pre-trained Transformer).
2. Similarly, there are transformers based on only the encoder block, for example, Google's BERT (Bi-directional Encoder Representations from Transformers).



# Transformers vs LLMs

Not all LLMs are transformers. LLMs can also be based on recurrence (or even convolution). [LSTM Based LLM](#)

Not all Transformers are LLMs. Transformers can also be used for Computer Vision. [Vision Transformers](#)

Have you used  
LLMs in your  
research?

And if yes, how?



# References:

- 1. [Hands-On Large Language Models](#)
- 2. [Building a Large Language Model \(from scratch\)](#)
- 3. <https://jalammar.github.io/illustrated-transformer/>



Thank you  
and  
Namaste!!

